Day 8 – Async Webhooks with Celery

🏷️ Teacher Prep & Classroom Setup

1. **Pre-class Checklist**:
    - Ensure you have Docker running on your system.
    - Run the Redis server in a container (`docker run -d -p 6379:6379 redis:7-alpine`).
    - Open a browser window ready to access the Django Admin panel.
    - Prepare two separate terminal tabs: one for running the management command, and one for running the Celery worker daemon.
2. **Prerequisites Checklist**:
    - Students must have completed Day 7 successfully (Synchronous webhook sending working with `requests.post`).
3. **Required Material**:
    - Whiteboard or slide showing the task queue broker architecture: `Producer (Management Command) → Message Broker (Redis Queue) → Worker (Celery Daemon) → database (Logs)`
    - Compare to a restaurant ticket system: the waiter (producer) drops tickets on the rail (broker), and the cook (worker) processes them asynchronously.

🎯 Learning Objectives

By the end of this class, students will be able to:

- Explain the role of a message broker (Redis) and a task runner (Celery) in async applications.
- Configure Celery inside a Django project environment.
- Create a `WebhookDeliveryLog` database audit trail.
- Build resilient asynchronous Celery tasks using automatic retries and exponential backoff calculations.
- Use `.delay()` to dispatch tasks asynchronously.
- Run and monitor Celery worker processes from the command line.

⏱️ Classroom Timeline (90 min)

Segment	Duration	Focus / Teacher Action
1. Hook & Big Picture	15 min	Define background worker models. Map the producer-broker-consumer relationship.
2. Key Concepts & Core Ideas	15 min	Explain shared tasks, bindings, exponential backoff, and late acknowledgments.
3. Live Coding Walkthrough	45 min	Start Redis, configure Celery settings, write the Audit Log model, implement the Celery task, update the CLI command, and run the worker.

4. Check for Understanding	10 min	Q&A on Redis message queue states, worker crashes, and task acknowledgment flags.
5. Class Wrap-up & Checkpoint	5 min	Verify async webhook tasks execute in the worker terminal and write logs to admin.

🚀 Lecture Step-by-Step Delivery Plan

1. Hook & Big Picture (15 min)

- **Teacher Action**: Open the queue diagram. Ask: *"What happens if a publisher's server goes down during our key check? In yesterday's code, the management command waited, blocked, and if the network request crashed, that webhook was lost forever. Today, we will throw the webhook job into a Redis buffer queue, let our management command finish instantly, and have a separate background process handle the HTTP delivery and automatic retries in the background."*
- **Decoupled Architecture**: Explain that by isolating the execution queue, if our web server crashes, tasks in Redis remain safe. If a publisher is offline, Celery will wait and try again later.

2. Key Concepts & Core Ideas (15 min)

Introduce these asynchronous concepts:

- **Task Queue / Broker**: An intermediary (Redis) that receives messages from the publisher and buffers them until a consumer (Celery worker) retrieves them.
- **`@shared_task`**: A decorator that defines a reusable Celery task without requiring explicit imports of the project-specific Celery application instance.
- **`bind=True`**: Binds the task function to the task instance, passing the task object itself as `self` (enabling us to invoke retries).
- **Exponential Backoff**: Instead of retrying immediately (when the remote server is likely still down), we double the wait time on each retry: `60 * (2 ** attempt)` (i.e. 60s, 120s, 240s).
- **Late Acknowledgment (`task_acks_late=True`)**: Enforces that the worker only tells Redis it completed the task *after* the function successfully executes. If the worker crashes mid-task, Redis re-queues the task for another worker.

---

3. Live Coding Walkthrough (45 min)

Step 3.1: Starting the Redis Message Broker

- **Explain**: Celery needs a queue to store tasks. We will spin up a lightweight Redis instance in a background Docker container.
- **Code**:

```
docker run -d -p 6379:6379 redis:7-alpine
```

- **Verify**: Verify that Redis is listening by running `docker ps` or connecting via `redis-cli ping`.

Step 3.2: Configuring Celery inside Django

- **Explain**: Create the Celery configuration file in the settings directory, update package initialization files, and add broker URLs to Django settings.
- **Code**: Create `gamekey_platform/celery.py`:

```
import os
from celery import Celery

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'gamekey_platform.s

app = Celery('gamekey_platform')
app.config_from_object('django.conf:settings', namespace='CELERY')
app.autodiscover_tasks()
```

Update `gamekey_platform/__init__.py` to load Celery on startup:

```
from .celery import app as celery_app

__all__ = ('celery_app',)
```

Add these configurations to the bottom of `gamekey_platform/settings.py`:

```
CELERY_BROKER_URL = 'redis://localhost:6379/0'
CELERY_RESULT_BACKEND = 'redis://localhost:6379/0'
CELERY_TASK_ALWAYS_EAGER = False  # set True in tests to run tasks
```

- **Verify**: Run `celery -A gamekey_platform inspect ping` to ensure Celery can connect to Redis.

Step 3.3: Creating the Webhook Audit Log Model

- **Explain**: In production, we must keep a permanent log of all webhook notifications, retries, and delivery responses for auditing and debugging.
- **Code**: Add `WebhookDeliveryLog` to `games/models.py`:

```
class WebhookDeliveryLog(models.Model):
    game_key = models.CharField(max_length=50)
    publisher = models.ForeignKey(Publisher, on_delete=models.CASCA
    payload = models.JSONField()
    response_status = models.IntegerField(null=True, blank=True)
    error_message = models.TextField(blank=True)
    attempt = models.IntegerField(default=0)
    success = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['-created_at']

    def __str__(self):
        status = 'OK' if self.success else 'FAIL'
        return f"[{status}] {self.game_key} attempt #{self.attempt}
```

Run the database migrations and register the model:

```
python manage.py makemigrations
```

```
python manage.py migrate
```

Register the model in `games/admin.py`:

```
from .models import WebhookDeliveryLog
admin.site.register(WebhookDeliveryLog)
```

- **Verify**: Log into Django Admin and verify that the `Webhook Delivery Logs` section is visible.

Step 3.4: Writing the Celery Task with Retries

- **Explain**: Create `games/tasks.py`. This task retrieves the publisher settings, builds the payload containing the current attempt count, signs it, and dispatches the POST request. If the HTTP call fails or raises an error, the task registers a failure log and schedules a retry with exponential backoff.
- **Code**: Create `games/tasks.py`:

```
import hmac
import hashlib
import json
import requests
from celery import shared_task
from .models import Publisher, WebhookDeliveryLog


@shared_task(bind=True, max_retries=3, default_retry_delay=60)
def send_expiry_webhook_async(self, publisher_id, game_key_str, gam
    publisher = None
    payload = {}

    try:
        publisher = Publisher.objects.get(id=publisher_id)

        payload = {
            "event": "game_key.expired",
            "game_key": game_key_str,
            "game_title": game_title,
            "expired_at": expired_at_iso,
            "attempt": attempt,
        }

        secret = publisher.webhook_secret.encode()
        body = json.dumps(payload, sort_keys=True).encode()
        signature = hmac.new(secret, body, hashlib.sha256).hexdiges

        headers = {
            "Content-Type": "application/json",
            "X-Signature": f"sha256={signature}",
        }

        response = requests.post(
            publisher.webhook_url,
```

```
                json=payload,
                headers=headers,
                timeout=5,
            )

            WebhookDeliveryLog.objects.create(
                game_key=game_key_str,
                publisher=publisher,
                payload=payload,
                response_status=response.status_code,
                attempt=attempt,
                success=response.status_code < 400,
            )

            if response.status_code >= 400:
                raise Exception(f"HTTP {response.status_code} from publ

        except Exception as exc:
            if publisher:
                WebhookDeliveryLog.objects.create(
                    game_key=game_key_str,
                    publisher=publisher,
                    payload=payload,
                    error_message=str(exc),
                    attempt=attempt,
                    success=False,
                )

            # Exponential backoff: 60s, 120s, 240s
            raise self.retry(exc=exc, countdown=60 * (2 ** attempt))
```

- ⚠ **Common Pitfalls**:
  - **Omitting `raise` on retry**: If you call `self.retry()` without raising it, the function execution doesn't stop, which can result in double logs. Always write `raise self.retry()`.

Step 3.5: Updating the Management Command

- **Explain**: Update `check_expired_keys.py` to dispatch tasks to the queue using `.delay()` instead of calling the function synchronously. Note that because we update the status field in bulk via `.update()`, we must materialize the queryset to a list *before* running the update.
- **Code**: Update `games/management/commands/check_expired_keys.py`:

```
from django.core.management.base import BaseCommand
from django.utils import timezone
from games.models import GameKey
from games.tasks import send_expiry_webhook_async


class Command(BaseCommand):
    help = 'Mark expired keys and dispatch async webhook notificati
```

```
    def handle(self, *args, **options):
        expired_keys = GameKey.objects.select_related('game__publis
            expires_at__lte=timezone.now(),
            status='active'
        )

        # Capture before update() clears the queryset
        keys_to_notify = list(expired_keys)
        count = expired_keys.update(status='expired')

        for key in keys_to_notify:
            send_expiry_webhook_async.delay(
                publisher_id=key.game.publisher.id,
                game_key_str=key.key_string,
                game_title=key.game.title,
                expired_at_iso=key.expires_at.isoformat(),
                attempt=0,
            )

        self.stdout.write(self.style.SUCCESS(
            f'Expired {count} keys. Webhook tasks dispatched to Cel
        ))
```

Step 3.6: Running the Asynchronous Stack

- **Explain**: Run the Celery worker process and trigger the expiration flow to watch the system in action.
- **Code**: In terminal tab 1, start the background worker:

  ```
  celery -A gamekey_platform worker --loglevel=info
  ```

  In terminal tab 2, trigger the expiration check command:

  ```
  python manage.py check_expired_keys
  ```

- **Verify**: Watch the logs in terminal tab 1. You should see the tasks received, logged, and executed asynchronously. Open Django admin and verify that corresponding `WebhookDeliveryLog` rows are created.

---

4. Check for Understanding (10 min)

Question: "What's in Redis right now, after `.delay()` is called but before the worker picks up the task?"

Answer: Redis contains a serialized JSON message (the task payload) representing the Celery task. This message includes the task name, task ID, arguments (like `publisher_id`, `game_key_str`), and other metadata, waiting in a list (acting as a queue).

Question: "What happens if the Celery worker crashes mid-task? Is the task lost?"

Answer: With default settings (late acknowledgment disabled), the task is acknowledged when the worker starts, meaning it could be lost if it crashes mid-execution. If late acknowledgment is enabled (`task_acks_late = True`), the task is only acknowledged *after* successful execution; if the worker crashes, the broker (Redis) will re-queue the task for another worker.

Question: "Why do we log *both* successful and failed deliveries to `WebhookDeliveryLog`?"

Answer: Logging both creates a complete audit trail. It allows developers to diagnose issues, prove to publishers that webhooks were dispatched, analyze error patterns, and keep track of execution history for debugging delivery problems.

Question: "What does `bind=True` do, and why do we need it for retries?"

Answer: `bind=True` binds the task function to the task instance, passing the task instance as the first argument (`self`). We need it because retrying requires calling the `.retry()` method on the task instance itself (`self.retry()`).

Question: "If `max_retries=3` and all retries fail, what happens to the task? How would we know?"

Answer: The task will be marked as `FAILURE` by Celery. An exception (`MaxRetriesExceededError`) will be raised and logged. We would know by checking Celery worker logs, monitoring systems, or checking the `WebhookDeliveryLog` in the database where the final log entry will show `success=False` and the final attempt count.

---

5. Daily Checkpoint & Wrap-up (5 min)

- **Verification Criteria**:
  - ○ The Celery background worker starts cleanly.
  - ○ Running `check_expired_keys` runs instantly without blocking for HTTP calls.
  - ○ The Celery worker console outputs task receipt and execution details.
  - ○ The `WebhookDeliveryLog` entries populate in the admin panel showing accurate response statuses and attempt numbers.